

Table 1: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup $>1\times$ means the denominator is faster; **bold** marks $>1.1\times$.

Program	ADT	SoA	End-to-end					Fold passes					Map passes				
	fields	bufs	Am (s)	Ai (s)	Sm (s)	Am/Sm	Ai/Sm	Am (s)	Ai (s)	Sm (s)	Am/Sm	Ai/Sm	Am (s)	Ai (s)	Sm (s)	Am/Sm	Ai/Sm
reduceNestedList	6	2	0.0871	0.0935	0.0074	11.74 $\times$	12.60 $\times$	0.0871	0.0935	0.0074	11.74 $\times$	12.60 $\times$	–	–	–	–	–

Table 2: Native PAPI metrics speedup summary across all passes. For each program and metric, speedup is AoS/SoA where each side is the sum of per-pass median counter values. $>1\times$ means SoA reports fewer events.

Program	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
reduceNestedList	11.68 $\times$	0.62 $\times$	6.31 $\times$	2.91 $\times$	62.87 $\times$	2.80 $\times$	6.13 $\times$
Geomean	11.68 $\times$	0.62 $\times$	6.31 $\times$	2.91 $\times$	62.87 $\times$	2.80 $\times$	6.13 $\times$

Table 3: Native PAPI metrics totals across all passes (baseline variants). Each entry is (Am/Sm), where Am and Sm are sums of per-pass median counter values.

Program	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
reduceNestedList	4.4e+08/ 3.7e+07	1.2e+07 /2.0e+07	1.9e+06/ 3.1e+05	2.3e+02/ 8.0e+01	1.1e+06/ 1.8e+04	2.2e+02/ 8.0e+01	6.9e+06/ 1.1e+06

Table 4: Baseline Gibbon comparison (times only). Shown variants: AoS-mut, AoS-imm, SoA-mut. Times are full executable end-to-end wall time per run (s), not pass-sum time. **Bold** marks the fastest time across shown variants.

Program	Am	Ai	Sm
reduceNestedList	16.28	23.06	8.102

Table 5: Baseline Gibbon comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut. Speedups are computed from full executable end-to-end wall time per run. $>1\times$ means the denominator is faster.

Program	Am/Sm	Ai/Am	Ai/Sm
reduceNestedList	2.01 $\times$	1.42 $\times$	2.85 $\times$

Table 6: Per-pass performance for `reduceNestedList`, ADT has 6 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
reduction	F	2/6	67%	0.0871 $\pm$ 0.00071	0.0935 $\pm$ 0.00005	0.0074$\pm$0.00001	11.74 $\times$	1.07 $\times$	12.60 $\times$
Total				0.0871	0.0935	0.0074	11.74 $\times$	1.07 $\times$	12.60 $\times$
Geomean						11.74 $\times$	1.07 $\times$	12.60 $\times$	

Table 7: Per-pass PAPI counters for `reduceNestedList` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
reduction	F	4.4e+08/ 3.7e+07	1.2e+07 /2.0e+07	1.9e+06/ 3.1e+05	2.3e+02/ 8.0e+01	1.1e+06/ 1.8e+04	2.2e+02/ 8.0e+01	6.9e+06/ 1.1e+06
Total		4.4e+08/3.7e+07	1.2e+07/2.0e+07	1.9e+06/3.1e+05	2.3e+02/8.0e+01	1.1e+06/1.8e+04	2.2e+02/8.0e+01	6.9e+06/1.1e+06